

No-Code Empire — Phase 1: Complete Execution Guide

Foundation Layer · Prompts 1, 4, 5, 29–33 + All Supporting Steps

How to use this guide. Every action below is described at the keystroke level. Nothing is assumed. Steps are numbered sequentially within each task. Estimated time is given for each task. Dependency locks are marked with  — do not proceed past one until it's resolved. Asset prompts (images, infographics, copy) are marked  Verification steps are marked  Warning flags are marked .

PHASE 1 AT A GLANCE

Order	Task	Est. Time	Blocker?
1	Domain & account setup	2–3 hrs	Yes
2	Supabase project creation	30 min	Yes
3	Run master schema (Prompt 1)	45 min	Yes
4	Seed 100 tools (Prompt 29)	1–2 hrs	Yes
5	Vercel config (Prompt 31)	1 hr	Yes
6	Webhook security (Prompt 5)	45 min	Yes
7	Site 1 Lovable build (Prompt 4)	3–4 hrs	Yes
8	Error handling system (Prompt 32)	1 hr	No
9	GDPR + affiliate disclosure (Prompt 33)	1 hr	Yes (legal)
10	Admin dashboard (Prompt 30)	2 hrs	No
11	Logo & brand asset creation	2 hrs	No
12	Author persona headshots	1 hr	No
13	OG image templates	1 hr	No
14	Social profile assets	1 hr	No
15	Screenshot & tool image pipeline	1 hr	No
16	End-to-end verification	1–2 hrs	Yes

PRE-PHASE CHECKLIST

Complete every item here before running a single prompt. These are the things that will block you mid-session if you don't have them ready.

Accounts to create (do all of these now)

1. Supabase

1. Go to `supabase.com` → click "Start your project"
2. Sign up with GitHub (recommended — simpler auth later)
3. Create a new organization: `nocode-empire`
4. Note your organization ID — you'll need it if you use the CLI later

2. Vercel

1. Go to `vercel.com` → "Sign Up" → connect with GitHub
2. Install the Vercel CLI: `npm install -g vercel`
3. Run `vercel login` in your terminal and authenticate
4. You do not need to create projects yet — that happens in Task 5

3. Lovable

1. Go to `lovable.dev` → create account
2. Subscribe to a paid plan — Agent Mode requires it
3. Confirm you can create a new project with Agent Mode enabled (the input field at the bottom of the project creation screen says "Agent Mode")

4. Resend

1. Go to `resend.com` → create account
2. Add and verify your sending domain immediately — DNS propagation takes 24–48 hours and will block newsletter launch if you wait
3. Copy your API key → store in your password manager as `RESEND_API_KEY`
4. Note: new Resend accounts are rate-limited to 100 emails/day. Request a limit increase immediately via Settings → Sending Limits

5. Google

1. Create a Google account dedicated to this project (or use an existing one)
2. Go to `search.google.com/search-console` → "Add property" → add your domain (you'll verify it via DNS once your domain is registered)
3. Go to `analytics.google.com` → create a new property for Site 1 → copy the Measurement ID (`G-XXXXXXXXXX`)
4. Go to `console.cloud.google.com` → create a new project → enable the "Google Search Console API" and "PageSpeed Insights API"

6. Domains

1. Register `nocode-reviews.com` (or your chosen domain for Site 1) via Cloudflare Registrar (cheapest, and you'll need Cloudflare anyway)
2. Register placeholder domains for Sites 2-10 now — they're cheap and the longer you wait the more likely they're taken. You don't need to build on them yet.
3. Suggested domain pattern: `nocode-[niche].com` or `[niche]tools.review`
4. Point all domains to Cloudflare nameservers immediately

7. n8n (you don't need this for Phase 1 but set it up now so it's ready for Phase 3)

1. Go to `n8n.io` → sign up for n8n Cloud Starter (\$20/mo)
2. Alternative: deploy self-hosted on Railway (`railway.app`) — \$5/mo, more control
3. Do not configure any workflows yet — just get an instance running

8. Anthropic API

1. Go to `console.anthropic.com` → create account → add credit card
2. Generate an API key → store as `ANTHROPIC_API_KEY`
3. Set a spending limit: \$50/month to start

Information to gather before you start

Create a text file called `phase1_env_vars.txt` on your desktop. You'll fill it in as you go. Never commit this file to git.

```
NEXT_PUBLIC_SUPABASE_URL=  
NEXT_PUBLIC_SUPABASE_ANON_KEY=  
SUPABASE_SERVICE_ROLE_KEY=  
WEBHOOK_SECRET=          # generate with: openssl rand -hex 32  
RESEND_API_KEY=  
RESEND_FROM_EMAIL=       # e.g. hello@nocode-reviews.com  
INDEXNOW_API_KEY=        # generate with: openssl rand -hex 16  
NEXT_PUBLIC_GA_MEASUREMENT_ID=  
NEXT_PUBLIC_SITE_URL=    # e.g. https://nocode-reviews.com  
ADMIN_EMAIL=  
ANTHROPIC_API_KEY=
```

Generate your `WEBHOOK_SECRET` now. Open your terminal and run:

```
bash  
  
openssl rand -hex 32
```

Copy the output into your `phase1_env_vars.txt`. This is a one-time operation — use the same secret across all API routes on Site 1.

Generate your `INDEXNOW_API_KEY`:

```
bash  
  
openssl rand -hex 16
```

TASK 1 — SUPABASE PROJECT CREATION

Time estimate: 30 minutes

Output: Live Supabase project, connection strings in hand

Steps

1.1 Go to `app.supabase.com` → click "New project"

1.2 Fill in:

- Organization: `nocode-empire`
- Project name: `site1-nocode-reviews`
- Database password: generate a strong one and save it in your password manager
- Region: **US West (Oregon)** — lowest latency for Vercel's default us-west-2 region
- Pricing plan: Free tier is fine to start

1.3 Click "Create new project" and wait 2–3 minutes for provisioning.

1.4 Once live, navigate to: **Settings → API**

1.5 Copy the following into your `phase1_env_vars.txt`:

- **Project URL** → `NEXT_PUBLIC_SUPABASE_URL`
- **anon public key** → `NEXT_PUBLIC_SUPABASE_ANON_KEY`
- **service_role secret key** → `SUPABASE_SERVICE_ROLE_KEY`

⚠ **The service_role key bypasses all Row Level Security. Never put it in any client-side code or commit it to git. It goes only into Vercel environment variables marked as "Server" scope.**

1.6 Navigate to: **Settings → Database**

Copy the **Connection string (URI)** under "Connection Pooling" — you'll need this if you run any server-side migrations later.

1.7 Navigate to: **Authentication → Providers**

Disable all social providers. Leave only Email enabled.

Under Email settings: turn OFF "Enable email confirmations" — you're handling double opt-in manually via your own webhook.

1.8 Navigate to: **Authentication → URL Configuration**

Set Site URL to your domain: `https://nocode-reviews.com`

Add to Redirect URLs: `https://nocode-reviews.com/confirm`

✅ **Verify:** You can see the project in the Supabase dashboard, the API keys are in your env file, and you can reach the SQL Editor tab without errors.

TASK 2 — RUN THE MASTER SCHEMA (Prompt 1)

Time estimate: 45 minutes

Depends on: Task 1 complete

🔒 **Every other task in Phase 1 depends on this.**

Background

The master schema creates 14+ tables, all indexes, RLS policies, and the seed data for categories and the Site 1 voice persona. The SQL from the build prompt (Section 2 of the Lovable prompt) is exactly what you run here.

Steps

2.1 In Supabase, navigate to **SQL Editor** → click "New query"

2.2 Give the query a name: `01_master_schema`

2.3 Paste the entire SQL block from Section 2 of the Lovable Agent Mode build prompt. This includes all CREATE TABLE statements, indexes, RLS policies, and the INSERT statements for categories and site_voices.

2.4 Before running, scan for these specific lines and confirm they're present:

- `create extension if not exists "pgcrypto";`
- `create extension if not exists "pg_trgm";`
- `create table tools (`
- `create table articles (`
- `create table subscribers (`
- `create table sponsored_slots (`
- `create table site_voices (`
- `insert into categories`
- `insert into site_voices`

2.5 Click "Run" (Ctrl+Enter / Cmd+Enter).

2.6 Watch for errors in the output panel. Common errors and fixes:

Error message	Fix
<code>extension "pgcrypto" already exists</code>	Safe to ignore — already installed
<code>relation "tools" already exists</code>	Drop and re-run, or add <code>CREATE TABLE IF NOT EXISTS</code>
<code>syntax error at or near "["</code>	You have a markdown code fence inside the SQL — remove it
<code>permission denied for schema public</code>	You're using the anon key in the SQL editor, not the service key — re-login
<code>column X of relation Y does not exist</code>	A generated column references a column that hasn't been created yet — check ordering

2.7 After a successful run, navigate to **Table Editor** in the left sidebar.

 **Verify all of these tables appear:**

- ☐ tools
- ☐ articles
- ☐ comparisons
- ☐ categories (click it — should have 10 rows from seed INSERT)
- ☐ subscribers
- ☐ affiliate_clicks
- ☐ sponsored_slots
- ☐ link_map
- ☐ keyword_queue
- ☐ rank_history
- ☐ cta_tests
- ☐ tool_submissions
- ☐ newsletter_issues
- ☐ distribution_log
- ☐ site_voices (click it — should have 1 row for 'site1' / Marcus Reeve)

2.8 Run this verification query in the SQL Editor:

```
sql

select table_name, pg_size_pretty(pg_total_relation_size(quote_ident(table_name))) as total_size
from information_schema.tables
where table_schema = 'public'
order by table_name;
```

You should see 14+ tables listed.

2.9 Verify RLS is enabled. Run:

```
sql

select tablename, rowsecurity
from pg_tables
where schemaname = 'public'
order by tablename;
```

Every table should show `rowsecurity = true`. If any show `false`, run `ALTER TABLE [tablename] ENABLE ROW LEVEL SECURITY;`

2.10 Create the `site_errors` table (not in the core schema — needed by Prompt 32):

```
sql

create table site_errors (
  id          uuid primary key default gen_random_uuid(),
  url         text,
  route       text,
  error_msg   text,
  payload     jsonb,
  user_agent  text,
  created_at  timestampz default now()
);

alter table site_errors enable row level security;
create policy "errors_insert_public" on site_errors for insert with check (true);
create policy "errors_admin_all" on site_errors for all using (auth.role() = 'service');
```

2.11 Create the `error_log` table (referenced in the API webhook routes):

```
sql

create table error_log (
  id          uuid primary key default gen_random_uuid(),
  route       text,
  error       text,
  payload     jsonb,
  created_at  timestampz default now()
);

alter table error_log enable row level security;
create policy "error_log_admin_all" on error_log for all using (auth.role() = 'service');
```

✅ **Final verification:** Navigate to Table Editor → categories → confirm you see 10 rows (website-builders, app-builders, automation, ecommerce, databases, ai-tools, cms, forms, analytics, membership).

TASK 3 — SEED 100 TOOLS (Prompt 29)

Time estimate: 1.5–2.5 hours

Depends on: Task 2 complete

🔒 **Site 1 renders nothing useful without this data.**

Steps

3.1 Open a new Lovable or Claude session (NOT the Agent Mode session you're using for the site build — keep that clean).

3.2 Send this prompt:

PROMPT 29: Create the complete seed dataset for the No-Code Empire tools database. I need 100 no-code tools across all 10 site categories: website builders, app builders, automation tools, database/backend tools, ecommerce platforms, AI integration tools, agency tools, analytics/SEO tools, design/UI tools, and solopreneur tools. For each tool provide: name, slug, tagline, website_url, affiliate_url (use realistic affiliate URL patterns like https://[tool].com?via=nocodereviews), category, rating_overall (1-5, be realistic — not everything is 4.5), rating_ease, rating_features, rating_pricing, rating_support, has_free_plan (boolean), starting_price_mo (numeric, 0 if free), pricing_tiers as JSONB in this exact format: [{"name":"Free","price_mo":0,"price_yr":0,"limits":{"key":"value"}}, ...], status ('active'). Output as a single SQL INSERT INTO tools (...) VALUES (...), (...) statement I can run directly in Supabase. Use exact column names from this schema: id (omit — let gen_random_uuid() handle it), slug, name, tagline, website_url, affiliate_url, category, pricing_model ('free'/'freemium'/'paid'/'open-source'), starting_price_mo, has_free_plan, pricing_tiers, rating_overall, rating_ease, rating_features, rating_pricing, rating_support, review_count (set to realistic values between 50-3000), is_featured (true for top 10 tools), status. Include these specific tools: Webflow, Framer, Bubble, Glide, Adalo, FlutterFlow, Squarespace, Wix, Shopify, Notion, Airtable, Zapier, Make, n8n, Pabbly, Softr, Stacker, Xano, Memberstack, Kajabi, Teachable, Podia, Ghost, Beehiiv, ConvertKit, Mailchimp, Typeform, Jotform, Tally, Outseta, Memberful, Thinkific, Gumroad, Lemon Squeezy, Stripe (no-code), Plausible, Fathom, Hotjar, Databox, Supermetrics, Figma, Penpot, Canva, Webflow (design), Framer (design), Relume, Finsweet, Memberstack, WeWeb, Tooljet, Appsmith, Directus, Supabase, PocketBase, Backendless, DronaHQ, Retool, Forest Admin, Budibase, plus additional tools to reach 100.

3.3 The AI will return a long SQL INSERT statement. Copy it entirely.

3.4 Before running it in Supabase: manually verify these 5 specific tools for pricing accuracy:

- **Webflow** — check webflow.com/pricing — confirm the CMS plan is \$23/mo (monthly) or \$19/mo (annual)
- **Bubble** — check bubble.io/pricing — confirm Starter is \$29/mo, Growth is \$119/mo
- **Shopify** — check shopify.com/pricing — confirm Basic is \$39/mo (may have changed from \$29)
- **Zapier** — check zapier.com/pricing — confirm free tier limit and Starter price
- **Squarespace** — check squarespace.com/pricing — confirm Personal plan price

⚠ **Stale pricing at launch is the #1 cause of Google HCU penalties on affiliate review sites. Verify before you load.**

3.5 Fix any outdated prices in the SQL directly.

3.6 In Supabase SQL Editor → New query → name it [02_tools_seed](#) → paste the INSERT → Run.

3.7 If you get a conflict error (`(duplicate key value violates unique constraint "tools_slug_key")`), run this first:

```
sql

truncate table tools restart identity cascade;
```

Then re-run the INSERT.

3.8 After a successful run:

```
sql

select count(*) from tools; -- should return 100
select category, count(*) from tools group by category order by count desc;
```

Verify there are tools in at least 8 categories, with no single category having more than 20 tools.

3.9 Check affiliate URLs are present:

```
sql

select name, affiliate_url from tools where affiliate_url is null or affiliate_url =
```

If any tools have null affiliate_url, update them:

```
sql

update tools set affiliate_url = website_url || '?via=nocodereviews'
where affiliate_url is null;
```

3.10 Mark the top 10 featured tools:

```
sql

update tools set is_featured = true
where slug in ('webflow','bubble','notion','zapier','shopify','framer','airtable','g
```

 **Verify:**

```
sql

select name, rating_overall, has_free_plan, starting_price_mo, is_featured
from tools
order by rating_overall desc
limit 15;
```

You should see recognizable tools with sensible ratings (no tool rated above 4.8, no tool rated below 2.5).

TASK 4 — VERCEL DEPLOYMENT CONFIG (Prompt 31)

Time estimate: 1 hour

Depends on: Domain registered, Supabase keys in hand

 **Required before any site can be deployed.**

Steps

4.1 Install the Vercel CLI if you haven't already:

```
bash

npm install -g vercel
vercel login
```

4.2 In your Vercel dashboard (`vercel.com/dashboard`), click "Add New Project".

4.3 For Site 1 (Lovable export → Vercel):

- Click "Import Git Repository"
- If using Lovable: first export the project from Lovable (Settings → Export → Download ZIP), then push to a new GitHub repo, then import that repo into Vercel
- Framework preset: **Next.js**
- Root directory: `./` (or wherever `package.json` lives)
- Build command: `next build`
- Output directory: `.next`
- Node.js version: **20.x**

4.4 Before clicking "Deploy", set all environment variables. In the Vercel project settings → Environment Variables, add each of the following. For each variable, set it for **Production**, **Preview**, and **Development**:

Variable	Value	Scope
<code>NEXT_PUBLIC_SUPABASE_URL</code>	your Supabase URL	All
<code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code>	your anon key	All
<code>SUPABASE_SERVICE_ROLE_KEY</code>	your service role key	Server only
<code>WEBHOOK_SECRET</code>	your generated secret	Server only
<code>RESEND_API_KEY</code>	your Resend key	Server only
<code>RESEND_FROM_EMAIL</code>	<u>hello@yourdomain.com</u>	All
<code>INDEXNOW_API_KEY</code>	your generated key	All
<code>NEXT_PUBLIC_GA_MEASUREMENT_ID</code>	G-XXXXXXXXXXXX	All
<code>NEXT_PUBLIC_SITE_URL</code>	<u>https://nocode-reviews.com</u>	All
<code>ADMIN_EMAIL</code>	your admin email	Server only
<code>ANTHROPIC_API_KEY</code>	your Anthropic key	Server only

⚠️ Do not mark `SUPABASE_SERVICE_ROLE_KEY` or `ANTHROPIC_API_KEY` as "All" environments — they must be Server only. Vercel prefixes client-accessible vars with `NEXT_PUBLIC_`. Any variable without that prefix is server-side only by default in Next.js.

4.5 Create the `vercel.json` file in your project root:

json

```
{
  "framework": "nextjs",
  "buildCommand": "next build",
  "devCommand": "next dev",
  "installCommand": "npm install",
  "regions": ["sfo1"],
  "headers": [
    {
      "source": "/api/(.*)",
      "headers": [
        { "key": "X-Content-Type-Options", "value": "nosniff" },
        { "key": "X-Frame-Options", "value": "DENY" }
      ]
    },
    {
      "source": "/(.*)",
      "headers": [
        { "key": "X-DNS-Prefetch-Control", "value": "on" }
      ]
    }
  ],
  "rewrites": [
    {
      "source": "/*:indexnow_key.txt",
      "destination": "/api/indexnow-key"
    }
  ]
}
```

4.6 Configure your custom domain in Vercel:

- Project Settings → Domains → Add `nocode-reviews.com` and `www.nocode-reviews.com`
- Vercel will show you DNS records to add

4.7 In Cloudflare (where your domain is registered):

- Navigate to DNS for your domain
- Add the records Vercel shows you:
 - For apex domain (`nocode-reviews.com`): Add an `A` record pointing to `76.76.21.21` (Vercel's IP)
 - For `www`: Add a `CNAME` record pointing to `cname.vercel-dns.com`
- ⚠️ Set Cloudflare proxy status to **DNS only (grey cloud)** for these records — Vercel handles SSL and CDN. A proxied record through Cloudflare will cause SSL conflicts.

4.8 Wait for DNS propagation (5-30 minutes) then verify SSL:

```
bash
curl -I https://nocode-reviews.com
```

You should see `HTTP/2 200` with a Vercel `x-vercel-id` header.

4.9 Add your domain to Google Search Console:

- Go to `search.google.com/search-console` → Add property → choose "Domain" type
- Copy the TXT record and add it in Cloudflare DNS
- Click "Verify"

✅ Verify:

- Vercel dashboard shows "Ready" deployment status
- Your domain resolves with HTTPS
- `vercel env ls` in terminal shows all environment variables

TASK 5 — WEBHOOK SECURITY LAYER (Prompt 5)

Time estimate: 45 minutes

Depends on: Site 1 codebase exists (Lovable export)

🔒 **All 7 API endpoints are vulnerable without this.**

Steps

5.1 Send this prompt in a Claude/Lovable session focused on the webhook security implementation:

PROMPT 5: Write the complete webhook security implementation for the No-Code Empire sites. Each site has these endpoints: POST /api/publish-article, POST /api/add-subscriber, POST /api/log-affiliate-click, POST /api/update-tool, POST /api/add-cross-link, GET /api/keyword-queue. All POST endpoints must validate an X-Webhook-Secret header against an environment variable WEBHOOK_SECRET. Write:

1. A validateWebhookSecret() middleware function in TypeScript for Next.js App Router API routes — it reads the X-Webhook-Secret header, compares it to process.env.WEBHOOK_SECRET using a timing-safe comparison (crypto.timingSafeEqual), and returns a 401 response if it doesn't match.
2. A standardized error response helper: createErrorResponse(code: string, message: string, status: number) that returns a NextResponse with {error: string, code: string} body.
3. Show how to use validateWebhookSecret in a sample /api/publish-article route.ts.
4. Show the n8n HTTP Request node configuration (JSON) that sends the correct X-Webhook-Secret header.
5. Show the curl test command that verifies a correct secret returns 200 and an incorrect secret returns 401.
6. Show the Vercel environment variable setup for WEBHOOK_SECRET.

5.2 Take the output and create this file in your project: `lib/webhook.ts`

typescript

```
// lib/webhook.ts - generated from Prompt 5 output
import { NextRequest, NextResponse } from 'next/server';
import { createHash } from 'crypto';

export function validateWebhookSecret(request: NextRequest): boolean {
  const secret = request.headers.get('x-webhook-secret');
  const expectedSecret = process.env.WEBHOOK_SECRET;

  if (!secret || !expectedSecret) return false;

  // Timing-safe comparison prevents timing attacks
  const secretBuffer = Buffer.from(secret);
  const expectedBuffer = Buffer.from(expectedSecret);

  if (secretBuffer.length !== expectedBuffer.length) return false;

  try {
    return require('crypto').timingSafeEqual(secretBuffer, expectedBuffer);
  } catch {
    return false;
  }
}

export function unauthorizedResponse() {
  return NextResponse.json(
    { error: 'Unauthorized', code: 'INVALID_WEBHOOK_SECRET' },
    { status: 401 }
  );
}

export function createErrorResponse(code: string, message: string, status: number =
  return NextResponse.json({ error: message, code }, { status });
}
```

5.3 Test the security layer with curl before wiring to n8n. Replace `your-secret` with your actual `WEBHOOK_SECRET`:

```
bash
```

```
# Should return 401
```

```
curl -X POST https://nocode-reviews.com/api/publish-article \
  -H "Content-Type: application/json" \
  -H "X-Webhook-Secret: wrong-secret" \
  -d '{"test": true}'
```

```
# Should return 400 (valid secret, invalid payload)
```

```
curl -X POST https://nocode-reviews.com/api/publish-article \
  -H "Content-Type: application/json" \
  -H "X-Webhook-Secret: your-actual-secret" \
  -d '{"test": true}'
```

5.4 Verify each API route imports and uses `validateWebhookSecret`. The pattern in every POST route:

```
typescript
```

```
// app/api/publish-article/route.ts
```

```
import { validateWebhookSecret, unauthorizedResponse, createErrorResponse } from '@'
```

```
export async function POST(request: NextRequest) {
  if (!validateWebhookSecret(request)) {
    return unauthorizedResponse();
  }
  // ... rest of handler
}
```

5.5 The `/api/log-affiliate-click` endpoint is an exception — it's called client-side from the browser and does NOT include the webhook secret. Instead, protect it via Supabase RLS (the INSERT policy already allows public inserts). Add a rate-limit check:

```
typescript
```

```
// In /api/log-affiliate-click/route.ts
```

```
// No webhook secret validation - this is called from the browser
```

```
// Rate limit: max 50 clicks per IP per hour (implement via Vercel KV or simple check)
```

```
export async function POST(request: NextRequest) {
  // No validateWebhookSecret() here
  const ip = request.ip || 'unknown';
  // ... proceed to insert
}
```

✓ **Verify:** All 6 POST endpoints return 401 when called without the correct secret. The GET (`/api/keyword-queue`) endpoint returns 401 without the secret (n8n sends it as a header even for GET requests).

TASK 6 — BUILD SITE 1 WITH LOVABLE AGENT MODE (Prompt 4)

Time estimate: 3–4 hours (mostly waiting for Lovable to build)

Depends on: Tasks 1–5 complete



The site must exist before Tasks 7–10 can be completed.

Steps

6.1 Open Lovable → New Project → enable **Agent Mode**.

6.2 Paste the entire content of your (`site1_lovable_build_prompt.md`) file (the output from the previous session) into the Agent Mode input field. All 13 sections. Do not split it.

6.3 While Lovable is building (it will take 20–40 minutes in Agent Mode), do not interrupt it. Watch the progress messages. If it asks a clarifying question, answer it — but the prompt was written to prevent this.

6.4 Common Lovable Agent Mode interruptions and how to respond:

Lovable asks	Your response
"Which database should I use?"	"Supabase — all credentials are in the environment variables section of the prompt"
"Should I use TypeScript?"	"Yes, TypeScript throughout"
"What UI component library?"	"shadcn/ui with Tailwind CSS as specified in the prompt"
"Should I generate placeholder content?"	"Yes, use the seed data from the SQL in Section 12"
"I can't access the Supabase database"	"That's expected — scaffold the code and I'll add the env vars after export"

6.5 When the build completes, click "Export to GitHub" (or the download option). You'll get a ZIP of the full Next.js project.

6.6 Unzip it, push to a new GitHub repository:

```
bash
```

```
cd site1-nocode-reviews
git init
git add .
git commit -m "Initial Lovable Agent Mode build"
git remote add origin https://github.com/yourusername/site1-nocode-reviews.git
git push -u origin main
```

6.7 Connect this GitHub repo to your Vercel project (if you haven't already):

- Vercel Dashboard → your project → Settings → Git → Connect to GitHub → select your repo

6.8 Trigger a deployment and watch the build log for errors. Common errors:

Error	Fix
Module not found: @/lib/supabase	Lovable used a different import path — search and replace
Type error: Property 'X' does not exist on type 'Y'	TypeScript type mismatch — check the database type definitions in <code>types/database.ts</code>
Error: NEXT_PUBLIC_SUPABASE_URL is not defined	Env var not set in Vercel — double-check Task 4.4
ReferenceError: window is not defined	SSR issue — wrap the offending code in <code>if (typeof window !== 'undefined')</code>

6.9 After a successful deploy, run through the build checklist from Section 13 of the build prompt. Test each page manually.

6.10 Fix any pages that didn't render correctly. The most common issues in Lovable builds:

- Fonts not loading: confirm Google Fonts import is in `app/layout.tsx`, not a component file
- Images returning 404: Supabase Storage bucket needs to be created and made public
- Admin dashboard redirecting to 404 instead of `/admin/login`: check that the middleware file exists at `middleware.ts` in the project root

6.11 Create the Supabase Storage bucket for images:

- Supabase Dashboard → Storage → Create bucket
- Name: `tool-assets`
- Public: Yes
- File size limit: 5MB
- Allowed MIME types: `image/jpeg, image/png, image/webp, image/svg+xml`

 Verify (minimum bar to pass Task 6):

- ☐ Homepage loads at your domain with tool grid visible
 - ☐ `/reviews/webflow` renders with tabs
 - ☐ `/admin/login` shows a login form
 - ☐ `/api/publish-article` returns 401 with no secret
 - ☐ Sitemap at `/sitemap.xml` returns XML
-

TASK 7 — ERROR HANDLING SYSTEM (Prompt 32)

Time estimate: 1 hour

Depends on: Task 6 complete

Steps

7.1 Send this prompt in your Lovable project or a code editor:

PROMPT 32: Build the complete error handling system for the No-Code Empire sites.

Deliverables:

1. A custom 404 page (`app/not-found.tsx` in Next.js App Router) that: shows a search bar with live Supabase search against the tools table, shows the top 6 most-viewed tools as small cards, shows a "Browse by category" grid linking to `/category/[slug]`, and never uses the word "error" — frames it as "We couldn't find that page".
2. A custom 500 error page (`app/error.tsx`) that logs to the `site_errors` Supabase table (columns: `url`, `error_msg`, `user_agent`, `created_at`) and shows a friendly message with a link home. Use the Supabase service role client for logging.
3. A universal form error handler hook: `useFormSubmit(endpoint: string, options: { onSuccess, onError, retries: number })` that: validates client-side before submitting, handles network failures with exponential backoff retry (max 3 attempts), shows specific field-level error messages using React state, and preserves all form input values in `localStorage` on network failure.
4. An API failure fallback component: `<APIFallback tool={toolName} />` that shows a static fallback message when an AI-powered tool (Should I Switch, Project Idea Validator) fails to get a response. The fallback shows: "Our AI is temporarily unavailable. Here are our top recommendations for [category]:" followed by 3 top-rated tools fetched from Supabase.
5. The `site_errors` and `error_log` table schemas (already created separately — just reference them). Design system: off-white (`#faf8f4`), dark ink (`#0f0e0d`), burnt orange (`#e8541a`), Instrument Serif headings, DM Sans body.

7.2 Integrate the output into your codebase:

- Replace the default Next.js `not-found.tsx` with the custom 404
- Add `error.tsx` to the app directory
- Add the `useFormSubmit` hook to `lib/hooks/useFormSubmit.ts`
- Add the `APIFallback` component to `components/ui/APIFallback.tsx`

7.3 Test the 404 page by navigating to a non-existent URL like `https://nocode-reviews.com/this-does-not-exist`.

7.4 Test the error logging by temporarily adding a throw to a test route, checking that it appears in the Supabase `site_errors` table.

✅ **Verify:** Custom 404 renders with search functionality. Error log table receives entries on simulated errors.

TASK 8 — GDPR + AFFILIATE DISCLOSURE (Prompt 33)

Time estimate: 1 hour

Depends on: Task 6 complete



Legal requirement — must be live before any content publishes or affiliate links go live.

Steps

8.1 Send this prompt:

PROMPT 33: Build the complete GDPR and affiliate compliance system for Site 1.

Deliverables:

1. A CookieConsentBanner component (position: fixed bottom, z-index 9999) that: appears on first visit via localStorage check ('cookie_consent' key), blocks the GA4 script from loading until consent is given (GA4 must NOT be in the page <head> — it must be loaded dynamically after consent), includes Accept All / Decline / Manage Preferences options, stores granular preferences: {analytics: boolean, marketing: boolean, functional: boolean}, has a minimal design matching the site: off-white #faf8f4, dark ink #0f0e0d, burnt orange #e8541a accept button, DM Sans 14px, dismisses with a smooth slide-down animation.
2. An AffiliateDisclosure component that renders a full-width amber callout: "This page contains affiliate links. If you click through and make a purchase, we earn a commission at no extra cost to you. This does not influence our reviews or ratings." — this component must appear: (a) at the top of every review page before the H1, (b) at the top of every comparison page, (c) at the top of every best-of list, (d) in the sticky sidebar CTA. Include the exact FTC-required disclosure language. Style: background #fef9c3 (amber-50), border-left 4px solid #d97706, padding 12px 16px, text #92400e, icon (i or Info icon from Lucide).
3. A PrivacyPolicy page at /privacy that covers: data collected (email addresses via newsletter signup, analytics via GA4 with consent, affiliate click events stored in Supabase), data processors (Supabase - database, Resend - email, Google Analytics - analytics, Anthropic API - AI features), user rights (access, deletion, portability), cookie policy (functional/analytics/marketing categories), and contact email for data requests. Write the full policy in plain English, not legalese.
4. A Terms of Service page at /terms covering: affiliate disclosure, no professional advice disclaimer, content accuracy disclaimer, DMCA policy, and acceptable use.
5. An email subscriber data disclosure (short, shown below the signup form): "By subscribing you agree to receive weekly emails. We never sell your data. Unsubscribe any time." with a link to /privacy.

8.2 Integrate into the codebase:

- Add `CookieConsentBanner` to `app/layout.tsx` — it must be ABOVE the GA4 script loader
- Add `AffiliateDisclosure` to the review page layout, comparison layout, and best-of layout
- Create `app/privacy/page.tsx` and `app/terms/page.tsx`
- Add privacy and terms links to the footer

8.3 Test the consent flow:

- Open the site in an incognito window
- Confirm GA4 is NOT firing (check browser Network tab for `gtag` calls — should be none)
- Click "Accept All"
- Confirm GA4 fires (check Network tab — you should see `https://www.google-analytics.com/g/collect` requests)
- Refresh the page
- Confirm the banner does not reappear

8.4 Verify PartnerStack TOS compliance: the affiliate disclosure must appear ABOVE THE FOLD (visible without scrolling) on any page that mentions a PartnerStack-affiliated tool (Airtable, Make, ClickUp, Glide, Softr, Stacker, Memberstack). The `AffiliateDisclosure` component at the top of the review page template handles this.

Verify:

- ☐ Cookie banner appears on first visit in incognito
 - ☐ GA4 does not fire before consent
 - ☐ GA4 fires after "Accept All"
 - ☐ Affiliate disclosure visible above fold on `/reviews/webflow`
 - ☐ `/privacy` and `/terms` pages render
 - ☐ Footer contains links to both pages
-

TASK 9 — ADMIN DASHBOARD (Prompt 30)

Time estimate: 2 hours

Depends on: Task 2 and Task 6 complete

Steps

9.1 Set up the admin user in Supabase Auth:

- Supabase Dashboard → Authentication → Users → "Invite user"
- Enter your admin email address
- Check your email and set a password via the magic link
- Note: this creates a user in Supabase Auth. The admin dashboard checks for a valid session.

9.2 The admin dashboard was built as part of Prompt 4 (the Lovable build). Verify it exists:

- Navigate to `https://nocode-reviews.com/admin`
- Should redirect to `/admin/login`
- Log in with your admin email and password
- Should land on the Overview tab

9.3 If the admin dashboard needs additional work beyond what Lovable built, send this supplement prompt:

PROMPT 30 (supplement): The admin dashboard at /admin already exists as part of the Site 1 build. I need to add these specific missing pieces:

1. The affiliate tracker table at /admin/affiliates — a sortable, filterable table with 20 affiliate programs pre-loaded (from the list in the spec). Columns: Program · Network · Commission · Status (dropdown: not applied / applied / approved / rejected) · Applied Date · Days Since Applied (calculated, red badge at 30+) · Affiliate URL (clickable link). Each row has a "Generate follow-up email" button that calls POST /api/ai/draft-email with the program name and returns a pre-written follow-up email in a modal.
2. The sponsored slots section at /admin/sponsored — shows all records from the sponsored_slots Supabase table with status badges (Active: green, Expiring <14 days: orange, Expired: red). Shows total monthly revenue from active slots. Has a "New Slot" button that opens a form to create a new sponsored_slots record.
3. A 30-day sparkline chart on the Overview tab using recharts — showing daily affiliate_clicks count for the last 30 days. Query: SELECT date_trunc('day', created_at) as day, count(*) FROM affiliate_clicks WHERE site = 'site1' AND created_at > now() - interval '30 days' GROUP BY day ORDER BY day. Design: off-white (`#faf8f4`), dark ink (`#0f0e0d`), burnt orange (`#e8541a`) accents.

9.4 Test all admin sections with your seeded data:

- Tools table should show all 100 tools from the seed
- Articles table should be empty (no articles published yet)
- Affiliates table should show 20 programs, all with status "not applied"
- Subscribers table should be empty
- Sponsored should show "No active slots"

9.5 ⚠️ Security check: Verify the admin route is protected. Open an incognito window and navigate to `https://nocode-reviews.com/admin/tools`. You should be redirected to `/admin/login`, not see the tools table.

Check that `middleware.ts` exists in your project root:

```
typescript

// middleware.ts
import { createMiddlewareClient } from '@supabase/auth-helpers-nextjs'
import { NextResponse } from 'next/server'
import type { NextRequest } from 'next/server'

export async function middleware(req: NextRequest) {
  const res = NextResponse.next()
  const supabase = createMiddlewareClient({ req, res })
  const { data: { session } } = await supabase.auth.getSession()

  if (req.nextUrl.pathname.startsWith('/admin') &&
    !req.nextUrl.pathname.startsWith('/admin/login')) {
    if (!session) {
      return NextResponse.redirect(new URL('/admin/login', req.url))
    }
  }

  return res
}

export const config = {
  matcher: ['/admin/:path*']
}
```

✓ Verify:

- ☐ `/admin` is inaccessible without login
- ☐ Login form works with your admin credentials
- ☐ Tools table shows seeded data
- ☐ Affiliates section shows 20 programs

TASK 10 — VISUAL IDENTITY & BRAND ASSETS 🎨

Time estimate: 2 hours

Depends on: Design system defined (from build prompt)

Tools needed: Midjourney, Adobe Firefly, DALL-E, or Canva for image generation; Figma or Canva for logo

This task has no corresponding prompt number — it's a gap in the original spec. Every item here is required before launch.

10.1 — Site Logo

Send this prompt to Midjourney, Adobe Firefly, or DALL-E:

"Minimalist wordmark logo for a tech review website called 'NoCode Reviews'. The name is set in a serif font (similar to Instrument Serif). Next to the text, a small geometric mark: a simple filled square with a small orange pixel or dot in the corner, suggesting a 'no-code' building block. Colors: dark near-black (● #0f0e0d) for the wordmark, burnt orange (● #e8541a) for the accent mark. Clean, professional, editorial in feel. White background. Vector-style render. No gradients, no 3D effects."

Generate 4 variants. Choose the one that reads clearly at 24px height (the header size). You need:

- SVG version (for web header) — if the AI can't output SVG, recreate in Figma or Canva
- PNG 200x50px (transparent background) — for email headers
- PNG 512x512px favicon source (the geometric mark only, no text)
- Favicon ICO at 16x16, 32x32, 48x48 (generate from the 512px PNG using (favicon.io))

Place these files:

- (public/logo.svg)
- (public/logo.png)
- (public/favicon.ico)
- (public/apple-touch-icon.png) (180x180)
- (app/icon.png) (32x32, Next.js App Router uses this for tab icon)

10.2 — OG Image Template

The dynamic OG image route ((/api/og)) generates images on the fly using (@vercel/og). But you also need a static fallback OG image for the homepage and pages without specific tool data.

Send this prompt to an image generator:

"Open Graph social preview image for a no-code tools review website. 1200x630px. Background: dark near-black (● #0f0e0d). Left half: large editorial heading text 'Find the right no-code platform.' in an off-white serif font. Below: smaller text 'Honest reviews of 100+ tools.' in a lighter gray. Right half: a stylized grid of 6 tool review cards (small rectangles with placeholder logos — abstract colored squares, not real logos), each with a small star rating in orange (● #e8541a). Top-right corner: site logo in white. Overall feel: clean, editorial, credible — not colorful or flashy."

Save as: (public/og-default.png) (1200x630px, under 1MB)

Update (app/layout.tsx) to reference this file in the default OpenGraph metadata:

```
typescript
```

```
export const metadata: Metadata = {  
  openGraph: {  
    images: ['/og-default.png'],  
  },  
}
```

10.3 — Author Persona Headshot (Marcus Reeve)

The Site 1 author persona (Marcus Reeve) needs a headshot for the About page and article bylines. This must look like a real person — not AI-generated, at first glance.

Send this prompt to Midjourney (or use a service like thispersondoesnotexist.com):

"Professional headshot photo of a male software developer / tech writer in his mid-30s. Slightly informal: he's wearing a dark henley or casual button-down, not a suit. He has medium-length dark hair, slight stubble. Looking directly at the camera with a confident but approachable expression. Background: blurred neutral office or coffee shop environment. The photo should feel like a LinkedIn profile picture or author bio photo — real and candid, not over-polished. Soft natural lighting."

Important: Generate at least 8 variants and pick the one that looks most like a real person. Run it through tineye.com to confirm it doesn't match any real person online.

Save as: public/authors/marcus-reeve.jpg (400x400px, square, under 200KB)

Update the author bio component and JSON-LD Person schema to reference this image path.

10.4 — Category Icons

Each of the 10 categories needs an icon. The schema already stores Lucide icon names, which the front-end renders. But you also need category hero images for category landing pages.

Send this prompt (10 separate generations, one per category):

"Minimal flat illustration icon for [category name] category on a no-code tools review website. Style: simple geometric shapes, 2-3 colors max. Primary color: burnt orange [#e8541a](#). Background: off-white [#faf8f4](#). No text, no gradients, no shadows. SVG-style render, 200x200px equivalent. Clean enough to work at 40x40px."

Categories to generate icons for:

1. Website Builders — "a simplified browser window with cursor"
2. App Builders — "a simplified mobile phone with building blocks"
3. Automation Tools — "two connected gears or a flow diagram arrow loop"
4. Ecommerce — "a simplified shopping bag"
5. Database Tools — "a simplified cylinder/database stack"
6. AI & Chatbot — "a speech bubble with a circuit pattern inside"
7. CMS Platforms — "a simplified document with edit cursor"
8. Form & Survey — "a simplified clipboard with checkboxes"
9. Analytics — "a simplified bar chart or trend line"
10. Membership & Community — "simplified group of 3 people silhouettes"

Save as: `public/categories/[slug].png` (200x200px each)

10.5 — Tool Screenshot Pipeline

Each tool in the database needs a `screenshot_url`. You can't manually screenshot 100 tools. Use this automated approach:

Step 1: Create a list of the 20 highest-priority tools (the featured + sponsored-candidate tools) and manually take screenshots:

- Visit each tool's website
- Take a full-browser screenshot at 1280x800px
- Crop to show the most interesting/representative part of the UI
- Save as `[tool-slug]-screenshot.jpg`

Step 2: Upload to Supabase Storage:

- Supabase Dashboard → Storage → `tool-assets` bucket → Create folder `screenshots`
- Upload all screenshots
- Copy the public URL for each

Step 3: Update the database:

```
sql

update tools set screenshot_url = 'https://[your-project].supabase.co/storage/v1/object/public/categories/[tool-slug]-screenshot.jpg';
```

For the remaining 80 tools, use this placeholder strategy until screenshots are added:

```
sql

-- Set a generic placeholder for all tools without a screenshot
update tools
set screenshot_url = 'https://[your-project].supabase.co/storage/v1/object/public/to
where screenshot_url is null;
```

Prompt for generating the placeholder screenshot image:

"A placeholder screenshot image for a no-code tool review website. 1280x800px. Shows a simplified, generic SaaS dashboard UI: a dark sidebar on the left with navigation items (Dashboard, Content, Users, Settings), a main panel with a header bar, a stats grid showing 4 metric cards (numbers and small chart icons), and a simple data table below. Color scheme: neutral grays and whites, subtle blue accents. Clean and professional. Not a real product — deliberately generic. Label 'Screenshot coming soon' in the center in gray text."

10.6 — Newsletter Header Image

The weekly newsletter needs a branded header image.

Prompt:

"Email newsletter header image for 'NoCode Reviews' weekly digest. 600px wide × 150px tall. Left side: site logo (text 'NoCode Reviews' in a serif font, with a small orange square accent mark). Center/right: the text 'Weekly Digest' in a lighter weight. Background: dark near-black (■ #0f0e0d). Text in off-white. Orange accent: (■ #e8541a). Clean, modern, editorial. No gradients."

Save as: `public/newsletter-header.png` (600x150px)

TASK 11 — ADDITIONAL PROMPTS FOR PHASE 1

The following prompts go beyond the original Phase 1 spec but are required before the site can operate properly. Run them in order.

Prompt A — `site_voices` Table Verification & Completion

The `site_voices` table was created in Task 2, but the n8n pipeline needs it fully populated before any content can be generated. The Lovable build seeded Site 1's voice only.

Prompt:

Create the complete site_voices INSERT statement for all 10 No-Code Empire sites. The table has columns: site (text, e.g. 'site1'), persona_name (text), persona_bio (text), writing_style (text), banned_phrases (text[]), few_shot_examples (text[]). For each site, create a distinct author persona:

- Site 1 (Platform Reviews): Marcus Reeve — already created, just confirm it's correct
 - Site 2 (Automation): a persona with 6+ years building automation workflows
 - Site 3 (Database/Backend): a persona with developer background who moved to no-code
 - Site 4 (Ecommerce): a persona who has built and sold 3 ecommerce stores
 - Site 5 (AI Integration): a persona who came from ML engineering into no-code AI tools
 - Site 6 (Agencies): a persona running a 5-person no-code agency
 - Site 7 (Analytics/SEO): a persona who is obsessive about data and measurement
 - Site 8 (SaaS Founders): a persona who has launched 2 no-code SaaS products
 - Site 9 (Design/UI): a designer-turned-no-code builder obsessed with aesthetics
 - Site 10 (Solopreneurs): a generalist who has tried 50+ tools building solo income streams
- Each persona needs: full name, 3-sentence bio, 2-sentence writing_style description, banned_phrases array (start with the standard list + 5 site-specific additions), few_shot_examples array of 2 sample paragraphs in that persona's voice. Make each voice genuinely distinct — different sentence lengths, different relationship to technical jargon, different level of cynicism.

Run the INSERT in Supabase SQL Editor.

Prompt B — Supabase Auth Admin User Hardening

Prompt:

Write the Supabase Auth configuration and Row Level Security setup that ensures only one admin email address can access the /admin dashboard. Specifically:

1. A Supabase Auth hook (using pg_net or a database function) that, on user signup, checks whether the email matches the ADMIN_EMAIL environment variable and if not, immediately marks the user as blocked.
 2. An RLS policy on the articles table that allows SELECT, INSERT, UPDATE, DELETE only for authenticated users whose email matches the admin email.
 3. A server-side check in the Next.js middleware that, in addition to checking session validity, verifies that session.user.email === process.env.ADMIN_EMAIL and redirects to /admin/login if it doesn't match.
 4. Instructions for rotating the admin email if needed.
-

Prompt C — Resend Email Templates

Prompt:

Write the complete Resend email template set for Site 1 — No-Code Platform Reviews. Using Resend's React Email format (react-email components), create:

1. Double opt-in confirmation email — Subject: "One click to confirm your NoCode Reviews subscription". Body: site logo, heading "Confirm your subscription", 1 sentence of what they'll receive, large confirm button linking to [SITE_URL]/confirm? token=[token], small disclaimer "If you didn't sign up, ignore this email."
2. Welcome email (sent after confirmation) — Subject: "You're in. Here's what to read first." Body: warm welcome, 3 featured article cards (tool name, one-line description, "Read →" button), one featured tool of the week (with affiliate link), unsubscribe link at bottom.
3. Weekly newsletter template — Subject: "[Tool of the week]: Is it worth it in [year]?" Body: header image, intro (1 short paragraph from Marcus Reeve), 3 article cards with excerpt and CTA, 1 sponsored slot section (conditional rendering — only shows if sponsored_slot data is provided), 1 featured tool comparison, unsubscribe + manage preferences link.
4. Admin notification email (sent to ADMIN_EMAIL when a new tool is submitted) — Subject: "New tool submission: [tool_name]". Body: submitted data in a clean table, "Review in admin →" button linking to /admin/submissions. Design system for all emails: off-white (□ #faf8f4) background, dark ink (● #0f0e0d) text, burnt orange (● #e8541a) buttons and headings, Instrument Serif for headings (with a web-safe serif fallback: Georgia), DM Sans for body (with Arial fallback). Max width 600px. Tested against Gmail, Outlook 2016+, Apple Mail.

Prompt D — IndexNow Key File & Robots.txt

These are small but commonly missed:

Prompt:

Write the implementation for:

1. The IndexNow key file endpoint at `/api/indexnow-key/route.ts` that returns the `INDEXNOW_API_KEY` environment variable as plain text with `Content-Type: text/plain`. This file must be accessible at `/[key-value].txt` on the domain (the `vercel.json` rewrite handles the URL mapping).
2. The `robots.txt` at `app/robots.ts` (Next.js App Router format) that: allows all crawlers, disallows `/admin/` and `/api/`, references the sitemap URL, and adds a `crawl-delay` of 1 for all bots.
3. A test that verifies the IndexNow key file is accessible by fetching `https://[domain]/[key].txt` and confirming the response body matches the key and the `Content-Type` is `text/plain`.

Prompt E — Affiliate Click Tracking Client Component

The `(POST /api/log-affiliate-click)` endpoint exists on the server, but you need a client-side component that wraps every affiliate link across the site.

Prompt:

Build the `AffiliateLink` React component for Site 1. This component wraps every affiliate link across the site and handles: (1) click tracking — on click, fires `POST /api/log-affiliate-click` with `{tool_id, article_id, site: 'site1', session_id (from sessionStorage, generated if absent), cta_position, cta_variant}` before navigating, (2) GA4 event — fires `gtag('event', 'affiliate_click', {tool_name, cta_position, cta_variant, page_location})` on click, (3) A/B variant assignment — reads the variant from `sessionStorage ('cta_variant')` or assigns one deterministically based on `session_id` hash, (4) opens the affiliate URL in a new tab with `rel="noopener noreferrer nofollow"`, (5) handles failures gracefully — if the `/api/log-affiliate-click` call fails, still navigates to the affiliate URL (never block a user's click due to tracking failure). Component props: `href` (string), `toolId` (string), `articleId` (string | null), `ctaPosition` (string), `children` (React.ReactNode), `className` (string). The component renders as a styled `<a>` tag that matches the design system button styles when passed a `className`. Export as a named component: `AffiliateLink`.

Prompt F — Sitemap Generator Implementation

Prompt:

Write the complete sitemap implementation for Site 1 using Next.js App Router's app/sitemap.ts format. The sitemap must:

1. Query the Supabase database server-side (using the service role client) for: all published articles (status='published'), all active tools (status='active'), all published comparisons (status='published'), all categories.
2. Generate URLs in this priority order with these settings:
 - Homepage: priority 1.0, changefreq 'daily'
 - Tool review pages (/reviews/[slug]): priority 0.9, changefreq 'weekly', lastmod = tool.updated_at
 - Comparison pages (/compare/[slug]): priority 0.8, changefreq 'monthly', lastmod = comparison.updated_at
 - Category pages (/category/[slug]): priority 0.7, changefreq 'weekly'
 - Best-of articles (/best/[slug]): priority 0.8, changefreq 'weekly', lastmod = article.updated_at
 - Tutorial/other articles: priority 0.6, changefreq 'monthly'
 - Static pages (compare, categories, about, submit, advertise, newsletter): priority 0.5, changefreq 'monthly'
3. Uses ISR revalidation — add export const revalidate = 3600 (1 hour).
4. Split into a sitemap index if total URLs exceed 10,000 (use generateSitemaps() for Next.js App Router).
5. Exclude all /admin/* routes, all /api/* routes, and any article with status != 'published'. Use the MetadataRoute.Sitemap return type from next.

TASK 12 — END-TO-END VERIFICATION

Time estimate: 1-2 hours

Depends on: All tasks above complete

Work through this checklist systematically. Do not mark an item as done until you have actually tested it in a browser, not just assumed it works.

Database Verification

sql

-- Run these queries in Supabase SQL Editor

```
select count(*) as tool_count from tools where status = 'active';           -- ex
select count(*) as category_count from categories;                         -- e
select count(*) as voice_count from site_voices;                           -- ex
select tablename from pg_tables where schemaname = 'public' order by tablename; -- e
select count(*) from tools where is_featured = true;                       -- ex
select count(*) from tools where affiliate_url is null;                     -- ex
```

API Endpoint Verification

Test each endpoint with curl. Replace `YOUR_SECRET` and `YOUR_DOMAIN`:

bash

1. Unauthorized request – should return 401

```
curl -s -o /dev/null -w "%{http_code}" \
  -X POST https://YOUR_DOMAIN/api/publish-article \
  -H "Content-Type: application/json" \
  -d '{}'
```

Expected: 401

2. Authorized request with invalid payload – should return 400

```
curl -s \
  -X POST https://YOUR_DOMAIN/api/publish-article \
  -H "Content-Type: application/json" \
  -H "X-Webhook-Secret: YOUR_SECRET" \
  -d '{}'
```

Expected: 400 with error message

3. Add subscriber – should return 200

```
curl -s \
  -X POST https://YOUR_DOMAIN/api/add-subscriber \
  -H "Content-Type: application/json" \
  -H "X-Webhook-Secret: YOUR_SECRET" \
  -d '{"email":"test@example.com","site":"site1","source":"test"}'
```

Expected: 200 {"success":true,"status":"pending_confirmation"}

4. Check subscriber was created

(Run in Supabase SQL Editor)

```
select email, status, confirm_token from subscribers where email = 'test@example.com'
```

Expected: 1 row, status='pending'

5. Keyword queue – should return empty array

```
curl -s \
  -H "X-Webhook-Secret: YOUR_SECRET" \
  "https://YOUR_DOMAIN/api/keyword-queue?site=site1"
```

Expected: {"keywords":[]}

6. Affiliate click log – no secret required, should return 200

```
curl -s \
  -X POST https://YOUR_DOMAIN/api/log-affiliate-click \
  -H "Content-Type: application/json" \
  -d '{"tool_id":null,"site":"site1","session_id":"test123","cta_position":"test","c'
```

Expected: 200 {"success":true}

Page Rendering Verification

Open each URL and confirm it renders correctly:

URL	What to check
<code>https://YOUR_DOMAIN/</code>	Hero loads, tool grid shows tools from DB
<code>https://YOUR_DOMAIN/category/website-builders</code>	Category tools render, count badge correct
<code>https://YOUR_DOMAIN/categories</code>	All 10 categories appear
<code>https://YOUR_DOMAIN/reviews/webflow</code>	All 6 tabs render, sidebar CTA appears
<code>https://YOUR_DOMAIN/compare</code>	Page renders (no comparisons yet)
<code>https://YOUR_DOMAIN/submit</code>	Form renders and submits
<code>https://YOUR_DOMAIN/advertise</code>	Page renders with pricing
<code>https://YOUR_DOMAIN/about</code>	Marcus Reeve bio and methodology
<code>https://YOUR_DOMAIN/privacy</code>	Full privacy policy renders
<code>https://YOUR_DOMAIN/terms</code>	Full terms of service renders
<code>https://YOUR_DOMAIN/tools/platform-wizard</code>	Wizard loads, can complete 10 questions
<code>https://YOUR_DOMAIN/tools/pricing-calculator</code>	Calculator renders with platform options
<code>https://YOUR_DOMAIN/tools/feature-matrix</code>	Tool selector renders
<code>https://YOUR_DOMAIN/tools/should-i-switch</code>	Form renders (AI call won't work without ANTHROPIC_API_KEY)
<code>https://YOUR_DOMAIN/admin</code>	Redirects to <code>/admin/login</code>
<code>https://YOUR_DOMAIN/admin/login</code>	Login form renders
<code>https://YOUR_DOMAIN/sitemap.xml</code>	Valid XML with at least 10+ URLs
<code>https://YOUR_DOMAIN/robots.txt</code>	Disallows <code>/admin/</code> and <code>/api/</code>
<code>https://YOUR_DOMAIN/[INDEXNOW_KEY].txt</code>	Returns the IndexNow key as plain text
<code>https://YOUR_DOMAIN/this-page-does-not-exist</code>	Custom 404 renders with search

SEO Verification

JSON-LD check on review page:

1. Go to `https://YOUR_DOMAIN/reviews/webflow`
2. View page source (Ctrl+U)
3. Search for `application/ld+json`
4. You should find 3 script blocks: SoftwareApplication, Review, FAQPage
5. Copy the SoftwareApplication JSON and paste into `validator.schema.org` — should validate without errors

Rich Results Test:

1. Go to `search.google.com/test/rich-results`
2. Enter `https://YOUR_DOMAIN/reviews/webflow`
3. Should show "SoftwareApplication" and "FAQPage" as eligible rich results

Open Graph check:

1. Go to `metatags.io` or `opengraph.xyz`
2. Enter your domain URL
3. Should show: title, description, and the OG image
4. If OG image doesn't show: confirm `/api/og` route is deployed and the static fallback `/og-default.png` exists

Performance Check

1. Go to `pagespeed.web.dev`
2. Enter `https://YOUR_DOMAIN/`
3. Run both Mobile and Desktop
4. **Target:** Desktop ≥ 90 , Mobile ≥ 70 (Mobile 80+ is aspirational for first launch)
5. If LCP > 3s: the most common cause is the hero section loading un-optimized images. Check that `<Image>` from `next/image` is used everywhere, not `<img>` tags.

GDPR/Consent Check

1. Open the site in an incognito window
 2. Open browser DevTools → Network tab → filter for `google-analytics.com`
 3. Verify: **no requests to google-analytics.com before you click "Accept All"**
 4. Click "Accept All"
 5. Verify: requests to `www.google-analytics.com/g/collect` now appear
 6. Refresh the page
 7. Verify: the cookie banner does not reappear
-

PHASE 1 COMPLETION CRITERIA

Phase 1 is complete when ALL of the following are true. Do not proceed to Phase 2 until every item is checked off.

Non-negotiable gates

- ☐ All 14+ Supabase tables created with RLS enabled
- ☐ 100 tools seeded with verified pricing
- ☐ Site 1 deployed and live at your domain with HTTPS
- ☐ All 7 API endpoints deployed and returning correct responses
- ☐ Webhook security returning 401 on unauthorized requests
- ☐ Admin dashboard accessible after login, inaccessible without
- ☐ GDPR consent banner blocking GA4 until consent
- ☐ Affiliate disclosure above-fold on all review pages
- ☐ Privacy policy and terms pages live
- ☐ Sitemap XML rendering at /sitemap.xml
- ☐ robots.txt disallowing /admin/ and /api/
- ☐ IndexNow key file accessible at /[key].txt
- ☐ JSON-LD schemas present on /reviews/[slug] pages
- ☐ All 4 interactive tools loading and functional
- ☐ GSC property verified with your domain
- ☐ GA4 measurement ID wired and receiving events (after consent)
- ☐ site_voices table populated for all 10 sites

Strong recommendations (not hard blockers, but do them)

- ☐ Author headshot uploaded and displayed on /about
 - ☐ OG image template renders at /api/og
 - ☐ All 100 tool screenshots populated (at least placeholders)
 - ☐ Resend domain verified (if not, newsletter launch is blocked)
 - ☐ All affiliate applications submitted (start the 2-4 week clock)
 - ☐ Error logging to site_errors and error_log tables working
-

WHAT COMES NEXT — PHASE 2 PREVIEW

Once Phase 1 is verified, Phase 2 addresses the SEO and technical infrastructure that must be in place before any content goes live at scale. The Phase 2 prompts (34-39) cover:

- **Prompt 34:** JSON-LD schema system (supplements what Lovable built — adds HowTo + Person schemas)
- **Prompt 35:** IndexNow + sitemap auto-regeneration (supplements the basic implementation from Phase 1)
- **Prompt 36:** Core Web Vitals optimization (run PageSpeed now to establish baseline)
- **Prompt 37:** E-E-A-T author persona system (adds the remaining 9 author pages + LinkedIn outlines)
- **Prompt 38:** Internal linking architecture (defines pillar/spoke structure before content fills in)
- **Prompt 39:** GA4 custom event tracking (verifies all 7 events fire correctly in DebugView)

Do not publish any articles yet. Phase 2 + Phase 3 must be complete before the content pipeline runs at scale. Publishing 40 articles/day without proper schema, E-E-A-T signals, and event tracking is how you waste your content budget.

The one action you CAN take before Phase 2 is complete: **submit all affiliate program applications today.** The approval timelines (2-4 weeks for ShareASale, PartnerStack, Impact) are the longest lead time in the entire project. Every day you wait is a day of live affiliate links you don't have.

Phase 1 estimated total: 18-24 hours across 2-3 working days.

Phase 2 estimated total: 6-10 hours.

First article should not publish until Phase 3 is complete.